Computer Science Review 56 (2025) 100724

Contents lists available at ScienceDirect

Computer Science Review

journal homepage: www.elsevier.com/locate/cosrev

Review article

A survey of heuristics for matrix bandwidth reduction

S.L. Gonzaga de Oliveira

Instituto de Ciência e Tecnologia, Universidade Federal de São Paulo, Av. Cesare Monsueto Giulio Lattes, 1201 - Eugênio de Melo, São José dos Campos, 12247-014, São Paulo, Brazil

ARTICLE INFO

Keywords:
Sparse matrices
Bandwidth reduction problem
Heuristics
Graph algorithm
Metaheuristics
Combinatorial optimization

ABSTRACT

This paper surveys heuristic methods for matrix bandwidth reduction, including low-cost methods and metaheuristics. This optimization min–max problem represents a demanding problem for heuristic methods. This paper poses the graph layout problem with its formal definition. The study also considers the application domains in which practitioners employ the linear graph layout problem on general matrices. Furthermore, this paper focuses on the techniques and procedures that provide excellent results and provides an extensive perspective of approaches to devise heuristics for matrix bandwidth reduction. Thus, this paper surveys the most significant research in the field and examines the current state-of-the-art heuristics for the bandwidth reduction problem.

Contents

1. Introduction 1
2. Problem formulation 2
3. Graph-theory algorithms for bandwidth reduction 3
   3.1. Application areas of graph-theory algorithms for bandwidth reduction 3
   3.2. A survey of graph-theory algorithms for bandwidth reduction 4
4. Metaheuristic algorithms for bandwidth reduction 5
   4.1. Application areas of metaheuristic algorithms for bandwidth reduction 5
   4.2. A survey of metaheuristic algorithms for bandwidth reduction 5
      4.2.1. Experiments with small-sized matrices 5
      4.2.2. Experiments with large-scale matrices 7
      4.2.3. Final considerations 7
   4.3. Examples of matrices to form a sparse matrix benchmark suite 7
5. Conclusions 9
Declaration of competing interest 9
Acknowledgments 9
Data availability 9
References 9

1. Introduction

The bandwidth reduction problem is a significant graph layout problem. The problem arises in several scientific and engineering applications, especially in finite-element discretizations. The bandwidth minimization problem of a symmetric matrix $A \in \mathbb{R}^{n \times n}$ is to approximate, as much as possible, all non-zero coefficients to the main diagonal of A.

The matrix bandwidth minimization problem possibly arose in the 1950s. Structural engineers analyzed the computational resolution of linear systems and were already concerned about the matrix bandwidth [1].

Harper [2] studied the bandwidth minimization problem in connection with error-correcting codes in 1964. The aim was to design error-correcting codes with minimum absolute average errors in a given class of graphs. Harper [2] determined the optimal labeling for the class n-cubes. Specifically, Harper [2] studied the problem of labeling hypercube vertices in n dimensions with integers from 1 to 2^n. Thus, the

E-mail address: sanderson.oliveira@unifesp.br.

https://doi.org/10.1016/j.cosrev.2025.100724
Received 1 August 2024; Received in revised form 1 December 2024; Accepted 11 January 2025
Available online 5 February 2025
1574-0137/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

summation over all pairs of adjacent vertices of the absolute differences of integers assigned to vertices was minimal. Harper [3] formally defined the problem in 1966.

The matrix bandwidth minimization problem (BMP) is $\mathcal{NP}$-Hard [4]. Papadimitriou [4] showed that determining whether the vertices of a graph have an ordering that produces a bandwidth of size b or less is $\mathcal{NP}$-Complete. The author reduced polynomially 3-SAT to the Linear Array Problem (LAP). Then, Papadimitriou [4] reduced polynomially LAP to the restricted LAP. So, Papadimitriou [4] reduced polynomially restricted LAP to BMP. Therefore, BMP is $\mathcal{NP}$-Hard. Thus, a typical algorithm for the problem is a heuristic method with the implication that it tries to find a labeling that yields a small bandwidth in an acceptable amount of time, even for large-scale instances [5]. Therefore, heuristic methods do not *minimize* bandwidth but *reduce* the matrix bandwidth.

Garey et al. [6] proved that the problem is $\mathcal{NP}$-Complete even for trees with maximum degree 3. Other researchers have identified many particular classes of graphs in which this problem remains $\mathcal{NP}$-Hard [7]. The bandwidth minimization problem is the GT40 problem in the book of Garey and Johnson [8] and the GT42 problem in the textbook of Ausiello et al. [9].

Williamson and Shmoys [10] defined an α-approximation algorithm for an optimization problem as a polynomial-time algorithm that, for all instances of the problem, generates a solution whose value is within a factor of α of the value of an optimal solution. Thus, one has proven that an approximation algorithm produces solutions within a factor of the optimum. In contrast, no research rigorously studied a heuristic that is not an approximation algorithm. Commonly, heuristics and metaheuristics are studied empirically, and they might perform well [10]. Thus, one can divide methods in this context into exact methods, approximation algorithms, heuristic methods (which are not approximation algorithms), and hybrid methods. Dueck and Jeffs [11] and Petit [7] provided excellent reviews of approximation algorithms and lower bounds for the problem. In this context, see the works of Cygan and Pilipczuk [12–15] and Fürer et al. [16]. On the other hand, this study surveys serial heuristics for matrix bandwidth reduction, including low-cost methods and metaheuristics.

Consider an undirected graph $G = (V, E)$ where V is a set of vertices, and E is a set of edges. All possible arrangements from V to $\{1, 2. \ldots, n\}$ comprise the search space for the bandwidth minimization problem, where $n = |V|$. There are $n!/2$ possible permutations of rows and columns for a symmetric matrix (associated with graph G) of order $n \times n$ because one can reverse each of the $n!$ permutations to yield an equivalent cost [17]. Therefore, checking every possible permutation for a sufficiently large matrix is impractical. Thus, since the 1960s, hundreds of heuristic methods have been proposed for the matrix bandwidth reduction problem [18,19]. Everstine [20], in 1979, cited 49 heuristic methods for reordering methods in this context. Gibbs [21], in 1980, informed that Everstine's list [20] was not exhaustive. Gibbs [21] presumed that there would be 50 to 100 reordering methods in this context until that year [22]. With a large number of heuristics for bandwidth reduction, there is evidence of the relevance of this problem [18,19].

This paper surveys heuristic methods designed for matrix bandwidth reduction. This study has four specific objectives: (1) The diverse application areas of bandwidth reduction are reviewed, including solving systems of linear equations. (2) The paper provides a history of various approaches to deal with this optimization problem. (3) The survey provides an overview of the research performed in matrix bandwidth reduction. (4) The paper summarizes the state-of-the-art heuristics (low-cost graph theory and metaheuristic algorithms) developed to solve the problem.

This article refers to a *graph-theory algorithm* in cases where the heuristic is a variant of the breadth-first search procedure [23]. This paper alludes to *metaheuristics* when a higher-level technique is applied to design a heuristic, i.e., a partial search algorithm. Examples of metaheuristics include Genetic Algorithm, Iterated Local Search (ILS), and Simulated Annealing, to name a few. This paper refers to a *metaheuristic algorithm* when the developer applies a metaheuristic to design a heuristic. In general, metaheuristic algorithms generate better outcomes than low-cost graph-theory heuristics at much higher running times.

Previous publications reviewed metaheuristic algorithms for the problem [18] and graph-theory algorithms for matrix bandwidth reduction [19]. These reviews relied on the Scopus and Google Scholar databases. The review performed by Torres-Jimenez et al. [24] corroborated the reviews published earlier [18,19]. Then, two new publications updated the review [25,26]. Later publications also updated the review (2019 [27,28], 2020 [29], 2022 [5], 2023 [30], and 2024 [31]). These papers also used the Scopus and Google Scholar databases to check whether the recently published algorithms outperformed state-of-the-art algorithms. The present study addresses only a fraction of the heuristics for bandwidth reduction already published. However, the main published heuristics are covered. Also, this survey covers the algorithms compared in each of the publications of these algorithms that became the state-of-the-art algorithm. Also, this survey discusses the main algorithms in the field, including those used as a basis for the design of variant algorithms that became state-of-the-art algorithms.

This paper organizes the subsequent sections as follows. This paper formulates the problem in Section 2. Afterward, Section 3 describes low-cost heuristics for bandwidth reduction. Subsequently, Section 4 presents metaheuristic algorithms for bandwidth reduction. Finally, Section 5 addresses the conclusions.

2. Problem formulation

The vertex labeling of an undirected graph is defined below.

Definition 1. A vertex labeling $S : V \to \{1, 2, \ldots, |V|\}$ of the undirected graph $G = (V, E)$ returns a number from the set $\{1, 2, \ldots, |V|\}$ for each vertex. So, $s(v)$ is the label of the vertex $v \in V$ in the vertex labeling $S = \{s(v_1), s(v_2), \ldots, s\left(v_{|V|}\right)\}$.

Practitioners employ identifiers in the interval $[1, |V|]$ to label the vertices in the set V in the vertex labeling S. So, S is a bijective mapping from V to the set $\{1, 2, \ldots, |V|\}$.

An application usually labels each vertex of a graph $G = (V, E)$ by a number in the interval $[1, |V|]$. Thus, the new labeling assigned by a bandwidth reduction heuristic is a permutation.

Consider the graph $G = (V, E)$ related to the matrix $A = [a_{ij}]$, such that the vertices $v, u \in V$ with labeling $s(v) = i$ and $s(u) = j$ are related to the rows i and j of the matrix A, respectively. Thus, $a_{ij} \neq 0$ if $\{v, u\} \in E$ and $a_{ij} = 0$ if $\{v, u\} \notin E$. Consequently, an undirected graph is associated with a symmetric matrix, and a directed graph is related to a non-symmetric matrix. So, matrices and graphs are employed interchangeably in this study. Designers of heuristics for bandwidth reduction represent the instances of the problem as graphs.

Fig. 1a shows a graph and its adjacency matrix on the left. Fig. 1b shows the graph in Fig. 1a after applying a heuristic for bandwidth reduction and the pattern of the corresponding adjacency matrix on the right. The six vertices in each graph are numbered from 1 to 6. Since the figure shows simple graphs, reflexive edges (edges from a vertex to itself, i.e., loops) are not permitted. Consequently, the diagonal entries of the subjacent adjacency matrix are all zero. However, practitioners replace the nonzero elements in the diagonal with pertinent information, e.g., the degree of the vertex. So, Fig. 1 represents the diagonal entries of the adjacency matrices in gray color. For example, reflexive edges are frequent in the matrices of the Suite Sparse Matrix Collection [32]. Particularly when solving differential equations, each entry in the diagonal of the matrix should be different from zero [33].

The linear array problem (LAP) and restricted LAP, mentioned in Section 1, are described as follows. LAP: given an undirected, weighted graph $G = (V, E)$ and an integer weight function p (on its edges), can one organize the vertices of the graph G in a linear array with unit

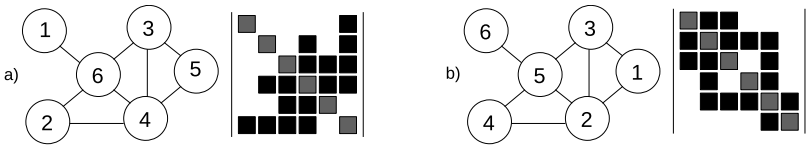


Fig. 1. (a) A graph and its adjacency matrix with the diagonal elements emphasized. (b) The graph after applying a heuristic for bandwidth reduction and the pattern of the corresponding adjacency matrix.

distances between consecutive vertices such that adjacent vertices do not have a distance higher than the weight of the edges that join them? Equivalently, does S exist such that $\{i,j\} \in E \implies |s(i) - s(j)| \leq p(\{i,j\})$, where $p(\{i,j\})$ is the weight of edge $\{i,j\}$? Restricted LAP: is a LAP in which the allowed weights (of the edges) are b and $2b-1$ for some positive integer b? The bandwidth $\beta_i(A_S)$ of the ith row of the symmetric adjacency matrix A_S is defined below.

Definition 2. Let $A_S \in \mathbb{R}^{n \times n}$ be a symmetric matrix, where $A = [a_{ij}]$, i.e., with entries a_{ij}, for $1 \leq i,j \leq n$. The bandwidth of the line i, $\beta_i(A_S)$, is $\beta_i(A_S) = 0$ if $(1 \leq j < i)\ a_{ij} = 0$ or $i = 1$; otherwise, $\beta_i(A_S) = i - \min_{1 \leq j < i}[j \mid a_{ij} \neq 0]$.

The definition indicates that the bandwidth $\beta_i(A_S)$ of row i of matrix A_S is the number of entries from the leftmost non-zero coefficient in row i to the main diagonal. Thus, $\beta_1 = 0$, $\beta_2 = \beta_6 = 1$, $\beta_3 = \beta_4 = 2$, and $\beta_5 = 3$ of the matrix on the right in Fig. 1. The definition indicates $(1 \leq i,j \leq n)\ a_{ij} = 0$ (i.e., null) if $i - j > \beta_i(A_S)$ for $i > j$. The bandwidth of a vertex in the subjacent graph $G_S = (V,E)$ of the matrix A_S is defined as follows.

Definition 3. The bandwidth of the vertex v of the undirected graph $G_S = (V,E)$ for a vertex labeling S is defined as $\beta_v(G_S) = 0$ if $\nexists u \mid \{v,u\} \in E \wedge s(u) < s(v)$; otherwise, $\beta_v(G_S) = \max_{\{v,u\} \in E \wedge s(u) < s(v)} [s(v) - s(u)]$, where $s(v)$ and $s(u)$ are the labels of the vertices v and u, respectively.

Bandwidth in the symmetric matrix is defined below.

Definition 4. Let $A_S \in \mathbb{R}^{n \times n}$ be a symmetric matrix, where $A = [a_{ij}]$, i.e., with entries a_{ij}, for $1 \leq i,j \leq n$. The bandwidth of the matrix A_S is $\beta(A_S) = \max_{1 \leq i \leq n} \left[\beta_i(A_S)\right]$.

The definition means that the bandwidth $\beta(A_S)$ is the highest number of entries, from the leftmost non-zero coefficient to the main diagonal, considering all n matrix rows. Therefore, the definition also means that the bandwidth of a symmetric matrix A_S is the number of sub-diagonals with non-zero coefficients to the left of the matrix's main diagonal. One can omit matrix A and the labeling S when the reader understands them in the context. With this, one writes only β. Thus, $\beta = 5$ ($\beta = 3$) in the matrix on the left (right) in Fig. 1. Bandwidth in the undirected graph is defined below.

Definition 5. Equivalent to the definition in matrices, the bandwidth of the undirected graph $G_S = (V,E)$, for a vertex labeling S, is $\beta(G_S) = \max_{v \in V}[\beta_v(G_S)]$.

BMP: given a graph G and an integer $b > 0$, is $\beta(G) \leq b$? The bandwidth minimization problem is also a particular case of LAP, with $(\forall \{i,j\} \in E)\ p[\{i,j\}] = b$ [4].

A banded matrix occurs when all non-null (or non-zero) coefficients lay around the main diagonal. Fig. 2 shows the pattern of a 494×494 non-banded matrix on the left. The figure shows the pattern of a 900×900 banded matrix, where the non-zero coefficients of the matrix are close to the main diagonal on the right.

With the bandwidth definition presented, the bandwidth of the symmetric matrix is $2\beta + 1$, consisting of the lower and upper portions of the band plus the main diagonal. Thus, the upper (lower) semi-band consists of all β entries of each row in the upper (lower) portion of the band, where $0 < j - i \leq \beta$ ($0 < i - j \leq \beta$). Hence, at most, β elements lay in each row (or sub-diagonals) in the semi-band of the matrix, i.e., from the main diagonal in the upper and lower portions of the matrix. Thus, β is the half-bandwidth rather than the matrix bandwidth. As a result, some authors define semi-bandwidth [34] or half-bandwidth [35] as having the same meaning as bandwidth shown in the previous definition. Likewise, the bandwidth is $2\beta + 1$. On the other hand, several papers [1,4,20,36–38] (to cite a few up to 1979) and other publications [21,39] (to cite a few up to 1981) used the nomenclature adopted in the present study.

Definition 6. The distance $d(v,u)$ between the vertices $v,u \in V$ is the shortest path length between them.

The eccentricity of a vertex and the graph's diameter are defined below.

Definition 7. The eccentricity $\ell : V \to \mathbb{N}$ of a vertex $v \in V$ is $\ell(v) = \max_{u \in V}[d(v,u)]$.

Definition 8. The diameter $\Phi(G) = \max_{v \in V}[\ell(v)] = \max_{v,u \in V}[d(v,u)]$ is the greatest eccentricity found in $G = (V,E)$.

Following Gibbs et al. [38], this paper defines a pseudoperipheral vertex as a vertex with eccentricity close to $\Phi(G)$.

Definition 9. A vertex v is a pseudoperipheral vertex if its eccentricity is close to the graph's diameter, i.e., $\ell(v) \approx \Phi(G)$.

3. Graph-theory algorithms for bandwidth reduction

As previously mentioned, graph-theory algorithms for bandwidth reduction rely on the breath-first search (BFS) procedure. Section 3.1 describes the motivation and application of graph-theory algorithms for bandwidth reduction, and Section 3.2 presents graph-theory algorithms for bandwidth reduction.

3.1. Application areas of graph-theory algorithms for bandwidth reduction

As previously mentioned, researchers realized that the order of the rows and columns of the linear system of type $Ax = b$ interferes with the solver's execution cost [1]. Thus, reordering algorithms reorder the rows and columns (or, equivalently, perform symmetric permutations on the vertices' labels of the subjacent graph) to reduce the linear system solver's running times. One technique for this is to reduce the bandwidth of matrix A. Reducing the bandwidth is a matrix pre-processing step. Practitioners perform it before executing the preconditioner and the iterative linear system solver.

Paging policies and the hierarchical memory architecture of modern computer designs favor programs that consider the locality of reference in memory. Therefore, bandwidth reduction benefits iterative linear system solvers. The reason is that the application has a greater chance of accessing adjacent vertices in the subsequent step. Thus, the system loaded them at the same level in the memory hierarchy. Heuristic methods for bandwidth reduction provide this spatial locality. Therefore, heuristics for bandwidth reduction are an alternative to obtaining

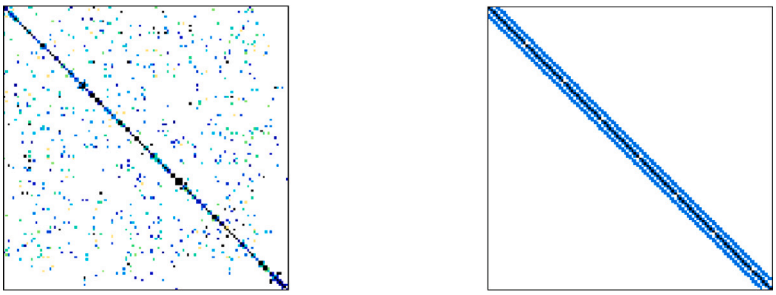


Fig. 2. The pattern of the matrices 494_bus and gr_30_30 taken from the SuiteSparse matrix collection [32].

vertex labeling in a way that yields spatial locality in memory [27].

The solution of large-scale sparse systems of linear equations is crucial in various application areas in Science and Engineering [40]. Most systems of linear equations appear from applying discretization methods (e.g., finite elements) to solve partial differential equations (PDEs) of elliptic and parabolic types [40]. On the other hand, large-scale sparse systems of linear equations also occur from applications not governed by PDEs [40]. Design and computational analysis of circuits, power system networks, processes in chemical engineering, economics models, and queuing theory are examples of applications not governed by PDEs that deliver large-scale and sparse systems of linear equations [22,40]. Also, bandwidth reduction is the first step in computing eigenvalues and eigenvectors in real or complex symmetric Hermitian matrices [41].

Thus, the bandwidth reduction problem has become the most studied graph layout problem, and developers have proposed several heuristics for the problem [18,19]. Therefore, heuristic methods for matrix bandwidth reduction are especially relevant in reducing the running times of linear system solvers. The resolution of linear systems of the type $Ax = b$, where A is a large-scale sparse matrix, is essential in various simulations in Science and Engineering and is, commonly, the step that requires the highest execution cost in the simulation [40]. Solving systems of linear equations accurately and efficiently is of great relevance for the viability and achievement of many software [34]. As previously mentioned, this subject is significant to scientists and engineers [22,34]. It is also substantial to several other users unfamiliar that a linear system solver runs at the core of the frequently used software [34].

The total cost of running the heuristic method for bandwidth reduction in conjunction with a preconditioner, plus the cost of the iterative linear system solver, must be less than the cost of executing only the solver in conjunction with the preconditioner, at least when using only a right-hand side vector [26]. Therefore, in the context of accelerating linear system solvers, the heuristic method for bandwidth reduction must have a low execution cost. Thus, practitioners use graph-theory algorithms for bandwidth reduction in this context [27].

3.2. A survey of graph-theory algorithms for bandwidth reduction

To the knowledge of the author of this study, Alway and Martin [35] presented the first algorithm for bandwidth reduction in 1965. It systematically changes rows and columns to reduce bandwidth until reaching the theoretical minimum. The algorithm searches for an unacceptable number of permutations.

After this first publication, researchers published many heuristics for bandwidth reduction [19]. This paper addresses the most relevant publications.

To this author's knowledge, Rosen [42] proposed the first published heuristic in 1968. The algorithm changes the labels of pairs of vertices at each iteration and can find a local minimum.

The Cuthill–McKee method [36], from 1969, traverses and labels the vertices in levels from the starting vertex. The greedy method sorts vertices adjacent to a vertex in increasing degree order.

George [37] realized, in 1971, that reversing the order of the Cuthill–McKee method reduces the profile $\sum_{v \in V} \beta_v(G_S)$ of a graph $G = (V, E)$ at least as satisfactory as the Cuthill–McKee method. The original Reverse Cuthill–McKee method (RCM) [37] used each vertex of minimum degree as the starting vertex. Thus, the method returned the best permutation found [38], generating a higher execution cost than labeling the vertices starting from a unique single initial vertex.

Gibbs et al. [38] designed an algorithm with two stages. In the first stage, the algorithm finds two pseudoperipheral vertices for global optimization. In the second stage, the algorithm internally uses the RCM method for local optimization. The GPS algorithm returns better bandwidth results than the RCM algorithm at a lower execution cost than the original RCM method [37]. The GPS algorithm outperformed previous algorithms for bandwidth reduction. As a result, the GPS algorithm became the state-of-the-art method in 1976.

Inspired by the pseudoperipheral vertex concept of the GPS algorithm, in 1979, George and Liu [43] published the current state-of-the-art algorithm for finding the initial vertex for the RCM method [44]. The RCM method with starting pseudoperipheral vertex given by the George-Liu algorithm has become the state of practice. The GPS algorithm yields better bandwidth results than this version of the RCM method but at a higher execution cost. In experiments in the acceleration of an iterative linear system solver, the RCM method returned better results than the GPS algorithm [26]. George and Liu [39] published in 1981 the RCM method with the George-Liu algorithm. Thus, the RCM method became the state of the practice low-cost algorithm for bandwidth reduction. The RCM method [39], with an initial vertex provided by the George-Liu algorithm [43], is implemented in several mathematical software, such as MATLAB® [45] and GNU Octave [46], with the function symrcm,[1] Wolfram Mathematica [47], ViennaCL [48], in the Boost C++ library [49],[2] SciPy [50], NetworkX [51], and spsrs from the Rust language [52]. Researchers proposed several graph-theory algorithms for bandwidth reduction in the following decades [19,25,26], but RCM [39] and GPS [38] dominated them [19].

Esposito et al. [53] proposed WBRA in 1998. WBRA is a graph-theory algorithm based on RCM. The authors compared their heuristic with TS-ET [54] (see Section 4.2.1) and the RCM method [39] available in MATLAB®. A publication [55] proposed a variant of WBRA.

Koohestani and Poli [56] achieved a breakthrough on this topic in 2011. Koohestani and Poli [56] evolved the RCM method by genetic programming, proposing the KP-band algorithm. In the training phase, Koohestani and Poli [56] used 25 matrices from different application areas and dimensions ranging from 24 to 960. Thus, the KP-band heuristic is a general-purpose algorithm similar to the RCM method.

[1] https://www.mathworks.com/help/matlab/ref/symrcm.html?requestedDomain=www.mathworks.com, https://octave.sourceforge.io/octave/function/symrcm.html.

[2] http://www.boost.org/doc/libs/1_38_0/libs/graph/doc/cuthill_mckee_ordering.html.

The KP-band algorithm yields overall bandwidth results and running times comparable to the RCM method [25–27,29]. Therefore, the RCM method remained the algorithm in the state of practice because it is simpler and better known. Koohestani and Poli's main contribution [56] was to propose, for the first time, a machine-generated heuristic for bandwidth reduction.

Two studies [25,26] from 2018 found that, with the great diversity of matrices arising from so many application areas using linear system solvers, there would be promising heuristics for bandwidth reduction per application area instead of just one state-of-the-art algorithm. Thus, a specific heuristic can speed up the linear system solver more than the RCM method in a distinguished application domain. As a result, a study [26] also found that an application does not need such a high bandwidth reduction to provide satisfactory results in accelerating a linear system solver in different application areas. Therefore, the study [26] used the Reverse Breadth-First Search procedure with the George-Liu algorithm (RBFS-GL) [26] to accelerate the linear system solver. This heuristic method is quite simple. Even so, RBFS-GL provides bandwidth reductions comparable to the RCM method at approximately half of the execution time [26,27,29]. In particular, the study [26] also found that the RCM, KP-band, and RBFS-GL heuristics outperformed the GPS algorithm [38] in accelerating a linear system solver. The GPS algorithm required more execution time than the three heuristics without providing much acceleration of the linear system solver.

In 2019, a study [27] proposed the RLK heuristic for bandwidth reduction. This heuristic is a hybrid variation of the RCM method and King's algorithm [57].

The Ant Colony Hyperheuristic (ACHH) system [27,29] evolved or selected RCM, KP-band, RBFS-GL, and RLK to generate new heuristics for bandwidth reduction specialized by application area. Thus, the resulting heuristics from the ACHH system are graph-theory algorithms. The publications [27,29] carried out tests with graphs with sizes ranging from 1,048,576 to 23,947,347 vertices and from 33,866,826 to 130,641,318 edges from the SuiteSparse Matrix Collection [32]. The heuristics returned by the ACHH system dominated the best graph-theory algorithms for bandwidth reduction in solution quality [29] and accelerating an iterative linear system solver [27]. If the application area of the practitioner is not one of the areas trained in the ACHH system [27,29], the user must employ the ACHH system to evolve the heuristics to a new expert-level set reordering algorithm specialized in the application area of the user. On the other hand, instead of performing a task that may not be simple, the practitioner can employ a general-purpose heuristic, such as the RCM method [39], which is the most successful algorithm for bandwidth reduction. On the other hand, the RBFS-GL heuristic is uncomplicated and faster than the RCM method, although the latter yields slightly better bandwidth results than the former.

4. Metaheuristic algorithms for bandwidth reduction

Section 4.1 describes the motivation and application of metaheuristic algorithms for bandwidth reduction. Section 4.2 presents metaheuristic algorithms for bandwidth reduction. Section 4.3 proposes a new sparse matrix benchmark suite.

4.1. Application areas of metaheuristic algorithms for bandwidth reduction

Unlike graph-theory heuristics for bandwidth reduction, metaheuristic algorithms for bandwidth reduction concentrate on areas that do not require a rigid limit on the execution cost of the heuristic method for bandwidth reduction. These areas include saving large hypertext media [58], load balancing on different processors in parallel implementations [59], small-world networks [60], network survivability [61], very-large-scale integration (VLSI) design [61], visual analysis of data sets using visual similarity matrices [62], graph entropy rate minimization [63], mesh layout optimization [64], symbolic model checking [65], and the seriation problem [66]. Also, the efficient representation of sparse matrices is crucial in big data applications. It is critical in this area to use a graph representation format with low requirements. In addition, applications must avoid unproductive computations and need to provide the data locality [67]. A format with low memory requirements allows large-scale instances of the problem to be solved [67]. Moreover, theory studies in the field encourage the development of heuristics that can reduce the bandwidth of large-scale matrices to a considerable proportion at a reasonable runtime. Thus, one can know how far the result yielded by a low-cost heuristic method is from the best solution already delivered in a particular instance [5,22,30].

Generally, metaheuristic algorithms provide better bandwidth results than graph-theory heuristic methods at a much longer running time [22]. Therefore, metaheuristic algorithms, such as VNS-band [68] and DRSA [24], are generally applied within an acceptable runtime on small-sized matrices with sizes up to 1000 vertices.

4.2. A survey of metaheuristic algorithms for bandwidth reduction

Authors have published many metaheuristic algorithms for bandwidth reduction [19]. This section covers the most relevant metaheuristic algorithms for bandwidth reduction. Section 4.2.1 covers metaheuristic algorithms designed for small-sized matrices. Section 4.2.2 describes metaheuristic algorithms designed for large-scale matrices. Section 4.2.3 provides the final considerations.

4.2.1. Experiments with small-sized matrices

Dueck and Jeffs [11] proposed a Simulated Annealing heuristic (SA-DJ) in 1995. The authors compared the results on 18 graphs with the GPS algorithm [38].

Esposito and Tarricone [54] proposed a heuristic based on the Tabu Search (TS-ET) metaheuristic in 1996. Martí et al. [69] proposed another heuristic based on the Tabu Search (TS) metaheuristic in 2001. The authors designed and tested a particular type of candidate list strategy to accelerate the selection of a move mechanism in the neighborhood of the current trial solution to search the solution space. The authors compared the results on 37 graphs with up to 199 vertices and 89 graphs with sizes up to approximately 1000 vertices with GPS [38] and SA-DJ [11]. The TS heuristic became the state-of-the-art metaheuristic algorithm in 2001.

Piñana et al. [70] proposed a heuristic based on the GRASP-PR metaheuristic in 2004. Each GRASP iteration creates a candidate solution with a procedure based on the GPS algorithm [38]. Afterward, it applies a local search procedure to find a local optimum. This local optimum is the final solution for the iteration. This intensification phase relies on critical vertices, a concept proposed by Martí et al. [69], i.e., vertices whose bandwidth is equal to the graph bandwidth. Piñana et al. [70] used the critical vertices decrease as an acceptance criterion for a label exchange between vertices. Performing multiple GRASP iterations samples the solution space. The authors also introduced a search procedure combining GRASP with path relinking. This procedure is a form of intensification of high-quality solutions previously found with the GRASP method. This approach yields new solutions by exploring trajectories that connect high-quality solutions to search the solution space for better solutions.

Piñana et al. [70] compared the results on 31 graphs with up to 199 vertices and 82 graphs with up to approximately 1000 vertices with GPS [38], TS [69], and GRASP. The GRASP-PR heuristic became the state-of-the-art method in 2004. The 113 matrices became the sparse matrix benchmark suite to compare metaheuristic algorithms in subsequent publications. The associated graphs have dimensions (ibm32) $32 \leq |V| \leq 991$ (jpwh_991) and (bcspwr01) $85 \leq |E| \leq 49,920$ (mbeacxc and mbeaflw). The reducible non-symmetric matrix sherman4 is the highest graph in this set with 1104 vertices, but the

highest strongly connected component in this digraph has only 546 vertices. The two other highest graphs are saylr3 and sherman1 with 1000 vertices, but the highest strongly connected components have only 681 vertices. Besides, these two matrices are identical in structure. They are only different in the weight of four edges, which is not significant for heuristics for bandwidth reduction. Thus, these two matrices are the same. Therefore, the highest connected graph is jpwh_991, with 991 vertices. In addition, the matrices mbeucxc and mbeaflw are also identical in structure. They are only different in the weight of their edges, which is not significant for heuristics for bandwidth reduction, as previously mentioned. Thus, these two matrices are the same. Hence, 111 instances with less than 1000 vertices comprise the testbed.

Lim et al. [61] proposed a genetic algorithm (GA) in 2006. Lim et al. [61] also proposed the Node Shift (NS) with the Hill Climbing procedure. The NS heuristic uses vertex label adjustments (shifts). NS arranges vertices in a sequence. Then, the heuristic shifts vertices to place closer in the sequence, those vertices whose label distance (i.e., their bandwidth) is the current graph bandwidth. This procedure requires only one step compared with the tabu search used by Esposito e Tarricone [54] and Martí et al. [69], where these heuristics require at least four swaps in similar routines [61]. The authors compared the heuristics on the 111 graphs compiled by Martí et al. [69,70] and another set composed of 14 graphs with GA [61], GRASP-PR [70], GPS [38], WBRA [53], and RCM (MATLAB®) [39]. The NS algorithm showed the best results and became the state-of-the-art metaheuristic algorithm in 2006.

Lim et al. [71] proposed a heuristic based on the Particle Swarm Optimization (PSO) metaheuristic in 2007. Still, in 2007, Lim et al. [72] proposed the NCHC and FNCHC heuristics. Although the authors did not mention this, these heuristics use concepts from the ILS metaheuristic. The Node Centroid (NC) routine locally adjusts the label of each vertex concerning its neighborhood, and a Hill Climbing (HC) local search generates improved outcomes. The authors also developed a fast version of the NCHC heuristic, denoted as FNCHC. The authors automatically adjusted the parameters according to the matrix size for this procedure.

Lim et al. [72] compared the results on the 111 graphs [69,70] with GPS [38], TS [69], GRASP-PR [70], GA [61], NS [61], and NCHC. NS yielded the best results, and FNCHC presented a computational cost almost as low as the GPS algorithm.

Rodriguez-Tello et al. [73] proposed another Simulated Annealing heuristic (SA) in 2008, called SA-δ. SA-δ randomly generates the initial solution. SA-δ starts at an initial temperature and, at each iteration, decrements it by a factor α. For each temperature, the maximum number of neighborhood labelings accepted and the maximum number of visited neighboring depend on the number of edges of the input graph. The reason is that denser graphs require a higher number of moves. The neighboring function relies on a rotation operator. The algorithm differentiates solutions by including a calculation (δ) that adds a fraction to the solution's objective function to escape from plateaus. SA-δ terminates when the current temperature reaches the final temperature or the bandwidth does not reduce. The authors compared the results on the 111 graphs [69,70] with TS [69], GRASP-PR [70], GPS [38], and SA-DJ [11], yielding the best results. A developer designs a metaheuristic algorithm for a specific suite benchmark. As previously mentioned, these 111 graphs are small-sized instances with vertex dimensions up to 1000. A faster metaheuristic algorithm can update the function objective only considering the vertices that had labels changed. The evaluation function δ seems to be expensive. Thus, SA-δ may be impractical for larger graphs because of its execution time.

Mladenović et al. [68] proposed the VNS-band heuristic in 2010. The VNS-band heuristic constructs an initial solution using a random breadth-first search, starting from a random vertex. The remaining steps of the VNS metaheuristic, search for a new candidate solution (called a perturbation step in the context of metaheuristics), local search, and neighborhood exchange, are followed until the stopping criterion is satisfied.

As previously mentioned, a constructive algorithm based on a variation of the breadth-first search (BFS) procedure starting from a random vertex provides the initial solution. BFS traverses the adjacency list of a vertex in the order of the input graph. Unlike BFS, the constructive algorithm proposed by Mladenović et al. [68] first labels a random vertex in the adjacency list and, subsequently, the other vertices in the order of the input graph in a rotational scheme. Maues et al. [74] evaluated several centrality orderings of the adjacency list in this constructive algorithm, and a random order yielded the best results.

Mladenović et al. [68] divided the VNS-band heuristic into four stages: initial solution, search for a new candidate solution (perturbation or shake), local search, and neighborhood exchange. After an initial solution $\mathfrak{S}$, the VNS-band heuristic initializes the radius k of the neighborhood of the initial solution $\mathfrak{S}$ with k_{min}. VNS-band performs changes in the candidate solution $\mathfrak{S}$ to try to "escape from the plateaus" in the solution space. In the search for a new candidate solution (perturbation stage), VNS-band finds a solution $\mathfrak{S}'$ in neighborhood $N_{k_{min}}(\mathfrak{S})$. The VNS-band heuristic searches for a new candidate solution, a step called perturbation or shake, by changing the labels of the vertices, starting from a solution $\mathfrak{S}$ and generating a solution $\mathfrak{S}' \in N_k(\mathfrak{S})$. VNS-band divides the search for a new candidate solution (perturbation step) into three procedures. The first procedure for searching for a new candidate solution is a method that previously selects some vertices with the highest bandwidths, and then their labels are changed. The second procedure for searching for a new candidate solution rotates the labels of some vertices. The third procedure selects which of the previous procedures searches for a new candidate solution.

A local search is applied to the candidate solution $\mathfrak{S}'$ to find a candidate solution $\mathfrak{S}''$. The local search of the VNS-band heuristic relies on the local search proposed by Lim et al. [61]. According to Mladenović et al. [68], the local search proposed by Lim et al. [61] presented better bandwidth results than the original local search used by the VNS metaheuristic.

The fourth step checks whether the candidate solution $\mathfrak{S}''$ is better than the current solution $\mathfrak{S}$. If it is, the candidate solution $\mathfrak{S}''$ becomes the current solution, and k is updated again to k_{min}. If not, the neighborhood radius k is increased by k_{incr}. In particular, VNS adjusts k_{incr} according to the problem.

VNS-band repeats these steps during a previously defined timeout. Mladenović et al. [68] established the VNS-band runtime as 500 s.

Mladenović et al. [68] compared the results on the 111 graphs with TS [69], GRASP-PR [70], SS-TS [75], GA [61], NS [61], PSO [71], and SA-δ [73]. Campos et al. [75] proposed a hybrid heuristic based on Scatter Search and Tabu Search (SS-TS) metaheuristics in 2011. Mladenović et al. [68] used the results from a preliminary paper's version in 2010.

With the tests, Mladenović et al. [68] showed that SA-δ [73] was the state-of-the-art metaheuristic algorithm in bandwidth reduction until 2010. With better results, the VNS-band heuristic became the state-of-the-art metaheuristic algorithm in 2010 in graphs with dimensions up to approximately 1000.

In literature reviews [18,25] developed in 2015, FNCHC [72] and VNS-band [68] were considered the most promising metaheuristic algorithms for bandwidth reduction. Afterward, Torres-Jimenez et al. [24] published the Dual Representation Simulated Annealing (DRSA) heuristic in 2015. DRSA is an improved implementation of SA-δ [73]. The main differences are as follows. DRSA uses three operators in the perturbation step, including the rotation operator (ROT) of the SA-δ heuristic. The REX operator changes the labels of two randomly chosen vertices. DRSA executes operator REX with a probability of 0.6. The NEX operator randomly finds a vertex v and changes its label with the label of a vertex adjacent to v, also chosen randomly. DRSA runs operator NEX with a probability of 0.2. DRSA executes the rotation operator with a probability of 0.2. Thus, the perturbation in the DRSA

heuristic comprises three satisfactory operators. In summary, there are two vertex label swap operators and a vertex label rotation operator.

Another relevant feature of the DRSA heuristic is the candidate solution evaluation function. In addition to the bandwidth β, the evaluation function uses the fractional part employed by SA-δ [73]. The inclusion of the fractional part allows better discrimination of candidate solutions. Better discriminating between candidate solutions allows the search process to move away from plateaus in the solution space. DRSA also improved the solutions encoding over SA-δ [73].

DRSA outperformed the VNS-band heuristic in reducing the bandwidth of very small matrices with dimensions up to approximately 1000. The authors compared the results on the 111 graphs with GRASP-PR [70], SA-δ [73], and VNS-band [68]. DRSA yielded better results than VNS-band, with approximately five times more running times to find the best results. With improved bandwidth reduction results, the DRSA heuristic has become the state-of-the-art metaheuristic algorithm on matrices with dimensions up to approximately 1000.

The initial temperature of DRSA is 1000, and the final temperature is 10^{-7}. The system temperature t is cooled gradually and slowly with $t \leftarrow t \cdot \alpha$, where $\alpha = 0.99$. This update is performed only in iterations without improvement in the solution. The initial size of the Markov chain is 40. Each time the algorithm cools the system, DRSA increases the size of the Markov chain, reaching $10 \cdot |V| \cdot |E|$ at the end of the algorithm for the input graph $G = (V, E)$. As a result, the running times of DRSA are long. A previous publication showed that DRSA is impractical for larger graphs [29].

The publication [24] showed the times in which the heuristic found the best solutions. In addition, in tests with a DRSA's implementation [29], DRSA does not return the solution shown in the publication [24] with just one execution running at many hours. Hence, one may need to run the heuristic several times to reach the results delivered by DRSA. Thus, the publication [24] shows the best solutions and the time of the solutions.

The relevant aspect is that DRSA [24] shows impressive bandwidth results and how a metaheuristic algorithm can reduce the bandwidth of the 111 graphs with sizes up to 1000 vertices [69,70]. As of 2015, developers published metaheuristic algorithms [28,31], but DRSA yields better bandwidth results.

4.2.2. Experiments with large-scale matrices

Gonzaga de Oliveira and Carvalho [5] proposed the ILS-band heuristic in 2022. The ILS-band heuristic comprises four stages: initial solution, perturbation strategy, local search, and acceptance criterion. A reverse breadth-first search procedure [26] generates the initial solution. Then, the ILS-band heuristic perturbs the current solution. Subsequently, the ILS-band heuristic employs a local search procedure to improve promising candidate solutions. An acceptance criterion determines whether the current candidate solution updates the best solution found so far.

ILS-band applies two perturbation strategies. The ILS-band heuristic executes the first perturbation at the beginning of each iteration. Then, it applies the local search routine if this current solution improves the best solution found so far. ILS-band runs the second more expensive perturbation only in a given number of iterations, which does not improve the current best solution found so far.

ILS-band was the first metaheuristic algorithm to return results in matrices with dimensions higher than one million at reasonable times (1200 s). However, the results were comparable with the GPS heuristic at higher running times.

Gonzaga de Oliveira and Carvalho [5] also proposed the FNCHC+ heuristic, a variation of the FNCHC heuristic [72]. The main improvement consists of only performing the expensive Hill Climbing local search for promising candidate solutions. Thus, FNCHC+ yielded satisfactory solutions to large-scale instances in a reasonable time [5].

The authors [5] compared the results on hundreds of small-sized matrices ranging from 5940 to 112,985 with GPS [38], ILS-band, FNCHC [72], and the VNS-band heuristic [68] since the DRSA heuristic is impractical for matrices of these sizes. GPS, FNCHC, ILS-band, and FNCHC+ dominated VNS-band. Furthermore, Gonzaga de Oliveira and Carvalho [5] compared the results on 76 large-scale matrices, ranging from one million vertices to 19 million vertices and from two million edges to 57 million edges, with GPS [38], ILS-band, and FNCHC [72], since the VNS-band heuristic [68] is impractical for matrices of these sizes. The FNCHC+ heuristic became the state-of-the-art metaheuristic algorithm for large-scale graphs, but the GPS [38] algorithm returned a lower execution cost than the FNCHC+ heuristic. Gonzaga de Oliveira [30] compared the FNCHC+ heuristic with the GPS [38] algorithm on 32 large-scale matrices, ranging from one million vertices to 24 million vertices and 34 to 131 million edges. The FNCHC+ heuristic outperformed the GPS algorithm in bandwidth reduction at lower execution costs than the GPS algorithm.

4.2.3. Final considerations

As mentioned, traditional metaheuristic algorithms for matrix bandwidth reduction, such as VNS-band [68] and DRSA [24], have a much higher execution cost than algorithms based on graph theory concepts [25,26,29,76–78]. Therefore, one can apply metaheuristic algorithms, such as VNS-band and DRSA, with satisfactory results at reasonable times to matrices with dimensions up to approximately 1000. On the other hand, the FNCHC heuristic [72] presented satisfactory results in medium-sized matrices with dimensions from 100 thousand to 1 million [25]. However, FNCHC did not yield satisfactory results for large-scale matrices with sizes above one million vertices [5]. In turn, the GPS algorithm presented the best results in large-scale matrices and was the state-of-the-art heuristic in this context until 2022 [5]. With this, Gonzaga de Oliveira and Carvalho [5] proposed the FNCHC+ heuristic, which surpassed the results of the GPS and FNCHC algorithms in large-scale matrices with sizes above one million vertices [5]. Thus, the FNCHC+ heuristic is considered the state-of-the-art metaheuristic algorithm for reducing the bandwidth of large-scale matrices. This heuristic method presents a higher execution cost than methods based on state-of-the-art graph theory concepts resulting from the ACHH system [29]. However, FNCHC+ delivers better bandwidth results than the resulting heuristics from the ACHH system [5]. Furthermore, the FNCHC+ heuristic presented better execution times than the GPS algorithm [30], the previous state-of-the-art method for reducing the bandwidth of large-scale matrices.

In summary, the DRSA heuristic [24] is considered the state-of-the-art metaheuristic algorithm for reducing the bandwidth of small-sized matrices with dimensions of approximately 1000. For large-scale matrices with sizes above one million, the state-of-the-art metaheuristic algorithm is the FNCHC+ heuristic [5].

4.3. Examples of matrices to form a sparse matrix benchmark suite

In 2001, Martí et al. [69,70] used the 111 matrices previously mentioned. Researchers used this collection of matrices as a benchmark suite to compare metaheuristic algorithms for bandwidth reduction. According to these publications, the authors extracted the matrices from the Harwell-Boeing Sparse Matrix collection. Duff et al. [79] developed this collection in 1992.

Piñana et al. [70] explained that Martí et al. [69] used 113 matrices in the tests. These matrices arose from different areas, and the SuiteSparse matrix collection [32], created in 2011, incorporated them. Developers used this set of matrices to benchmark metaheuristic algorithms over the last 24 years. However, the 111 graphs are from the 1970s and 1980s and are very small for today, with sizes ranging from 32 to 991 vertices and from 85 to 49,920 edges. For example, the will57 matrix with 57 vertices and 281 edges is from 1970. In addition, these matrices are from different application areas. However, unlike low-cost heuristics for accelerating iterative linear system solvers, metaheuristic algorithms for bandwidth reduction are only practical for

Table 1
Twenty-three matrices from the SuiteSparse matrix collection [32].

Matrix	Group	Year	$\|V\|$	$\|E\|$	Kind
MISKnowledgeMap	Moqri	2015	2,427	57,022	Undirected
har_10NN	ML_Graph	2020	10,299	151,736	graph
kmnist_norm_10NN			10,000	156,932	
Fashion_MNIST_norm_10NN			10,000	158,304	
worms20_10NN			20,055	240,826	
k49_norm_10NN			38,547	618,158	
power	Newman	1998	4,941	13,188	
smallworld	DIMACS10	2011	100,000	999,996	Random
vsp_befref_fxm_2_4_air02			14,109	196,448	undirected
vsp_barth5_1Ksep_50in_5Kout			32,212	203,610	graph
vsp_sctap1-2b_and_seymourl			40,174	281,662	
vsp_south31_slptsk			39,668	379,828	
vsp_model1_crew1_cr42_south31			45,101	379,952	
vsp_bump2_e18_aa01_model1_crew1			56,438	601,602	
vsp_vibrobox_scagr7-2c_rlfddd			77,328	871,172	
vsp_finan512_scagr7-2c_rlfddd			139,752	1,104,040	
vsp_msc10848_300sep_100in_1Kout			21,996	2,442,056	
vsp_c-30_data_data			11,023	124,368	
vsp_data_and_seymourl			9,167	111,732	
vsp_p0291_seymourl_iiasa			10,498	107,736	
preferentialAttachment			100,000	999,970	
boyd1	GHS_indef	2004	93,279	1,211,231	Optimization
lp1	Andrianov	2006	534,388	1,643,420	problem

the application areas commented on in Section 4.1. Also, 17 of these 111 graphs contain several strongly connected components, and the heuristic commonly uses the largest strongly connected component. Thus, the sizes of the test graphs are even smaller. Therefore, there is a need to propose a new sparse matrix benchmark suite for metaheuristic algorithms for bandwidth reduction.

Petit [80] made several relevant considerations for creating a set of benchmark matrices. Ideally, the test suite should contain matrices from real-world applications that the optimization problem formulates. Thus, a relevant decision is to incorporate into the test suite matrices from the application areas mentioned in Section 4.1.

In addition, the matrices should be easily obtained in different formats from a well-known database by the community. Thus, choosing only matrices contained in the SuiteSparse matrix collection [32] is an important decision.

An important decision is to incorporate matrices substantially higher than the Martí and co-authors' 111 matrices [69,70]. Thus, slow metaheuristic algorithms can be considered impractical.

Following Petit [80], another important decision is that random graphs comprise the test suite because such graphs may be amenable to probabilistic analysis. Consequently, one can draw general conclusions from the study.

Another decision is to include connected graphs in the test suite. When incorporating disconnected graphs in the test suite, one should describe the size of the highest strongly connected component of the graph. Thus, developers would compare the algorithms correspondingly.

Another relevant decision is to compose the test suite with at least 20 matrices. Hence, one can draw general conclusions from the study.

Table 1 shows 23 graphs from the SuiteSparse matrix collection [32]. Specifically, it shows the undirected graph MISKnowledgeMap (from 2015). This undirected graph arose from the visual analysis of data sets using a visual similarity matrix.

Table 1 shows five undirected graphs from machine learning datasets (from 2020). It also shows the power and small-world undirected graphs from small-world networks.

Consider the graph $G_S = (V, E)$, a vertex $v \in V$, and the sets $L(v, G_S) = \{u \in V : s(u) \leq s(v)\}$ and $R(v, G_S) = \{u \in V : s(u) > s(v)\}$. Graph $G_S = (V, E)$ is bipartite into the sets $L \subset V$ and $\bar{L} = R \subset V$. The sets are a vertex cut (or separation) at the vertex $v \in V$, with a label (or position) $s(v)$. The number of vertices in the set L adjacent to a vertex in the set R is $\mathfrak{v}(v, G_S) = |\{u \in L(v, G_S) : \{u, w\} \in E \wedge \exists w \in R(v, G_S)\}|$. The Vertex Separation Problem (VSP) $\min \max_{v \in V} \mathfrak{v}(v, G_S)$ is a min–max graph layout problem in which one wants to minimize the maximum $(\forall v \in V)\ \mathfrak{v}(v, G_S)$. Besides its theoretical significance as a canonical $\mathcal{NP}$-Hard problem [81,82], VSP formulates many relevant real-world problems, such as the VLSI design. The problem is $\mathcal{NP}$-Hard, even in some classes of restricted graphs [83]. Thus, Table 1 shows 12 random undirected graphs from 2011 arising from the Vertex Separation Problem.

Freire et al. [67] enhanced the sparse matrix storage of the graphs preferentialAttachment, boyd1, and lp1 using a heuristic for bandwidth reduction. The suite benchmark in Table 1 contains graphs from six different groups, indicating its diversity. Twenty-one small-sized graphs (i.e., until 100,000 vertices) and two medium-sized graphs (vsp_finan512_scagr7-2c_rlfddd and lp1) comprise the testbed.

Sometimes, the execution time of a metaheuristic algorithm for bandwidth reduction depends more on the number of edges than on the number of vertices in the input graph. Thus, one could consider the graphs small-world, vsp_finan512_scagr7-2c_rlfddd, vsp_msc10848_300sep_100in_1Kout, preferentialAttachment, boyd1, and lp1 too large for a metaheuristic algorithm. Specifically, one could consider them very large because a metaheuristic algorithm needs much time to search for a satisfactory solution in the solution space. On the other hand, these are real and recent matrices (2004, 2006, and four from 2011) from the SuiteSparse matrix collection, arising from practical applications in which the heuristic runtime for bandwidth reduction does not have a hard limit. Therefore, these are the instance sizes that metaheuristic algorithms must deal with in practice. Thus, these are challenging matrices for metaheuristic algorithms for bandwidth reduction. Precisely, a benchmark suite must be composed of difficult instances. Tests that include these matrices may show that slow algorithms are impractical [5,30]. Furthermore, the execution time of a metaheuristic algorithm for bandwidth reduction depends more on the graph structure than the number of edges and vertices. Thus, a bandwidth reduction heuristic may have longer running times on small matrices with complex structures than on large matrices with simple structures [5,30].

A computational experiment requires multiple runs with each heuristic on each matrix, e.g., ten runs. On the other hand, a computational experiment with metaheuristic algorithms with high execution costs requires at least three executions of each metaheuristic algorithm

on each matrix. Therefore, the computational experiment should show the average results of the executions, standard deviation, the best result, and the total runtime in which the metaheuristic algorithm executes, as this is the time the user waits for the solution.

Determining a reasonable timeout for a metaheuristic algorithm is relevant. In practice, the user does not wait only for the time the algorithm yields the best solution but the total runtime of the algorithm. Waiting for hours for a heuristic for bandwidth reduction to return the solution may not be reasonable. Thus, a publication [30] set 1500 s as the timeout of heuristics for bandwidth reduction to deliver solutions in graphs with sizes ranging from 1,048,576 to 23,947,347 vertices and from 33,866,826 to 130,641,318 edges.

5. Conclusions

This paper surveyed heuristic methods for the bandwidth reduction problem. The objective is to minimize the highest absolute difference of labels between all adjacent vertices of the graph. This survey included the most significant heuristic methods for the min–max problem. The extensive diversification of heuristic methods reviewed provides an overview of approaches that produce excellent bandwidth results. Developers can employ the expertise to conceive new heuristics for the linear graph layout problem. Also, one can utilize the techniques for other optimization problems.

The RCM method [39] is the most successful algorithm for the problem. It remains the state-of-the-practice low-cost heuristic for bandwidth reduction to accelerate iterative linear system solvers. The resulting heuristics from the ACHH heuristic [27,29] are currently state-of-the-art low-cost algorithms to accelerate linear system solvers. The RBFS-GL heuristic [26] delivers bandwidth results comparable to the RCM method, with approximately half the execution time.

Finding new low-cost methods by reinforcement learning is promising [84]. In this research topic, the developer must test the new heuristic to accelerate an iterative linear system solver, such as the zero-fill incomplete Cholesky-preconditioned conjugate gradient method [85].

The FNCHC+ heuristic [5,30] is currently the state-of-the-art metaheuristic algorithm for the bandwidth reduction of large-scale matrices, i.e., with sizes above one million and one hundred million edges. Developers should design better constructive methods than those used in the metaheuristic algorithms described. Furthermore, the developer should design better perturbation operators than REX, NEX, and ROT of the DRSA heuristic. Thus, the heuristic can better explore the solution space. However, the perturbation mechanism must be fast. Hence, one can apply the heuristic to large-scale instances. Also, a local search is a procedure that should be employed to find a local optimum. Likewise, this intensification phase should be of high quality and fast. The biggest weakness of FNCHC+ seems to be the lack of a fast local search procedure. With this, researchers will likely propose new powerful metaheuristic algorithms to overcome the FNCHC+ heuristic.

The DRSA heuristic [24] is currently the state-of-the-art metaheuristic algorithm for the bandwidth reduction of matrices with sizes up to approximately 1000. However, the 111 matrices used as testbeds are outdated to the present day. Therefore, the present paper showed examples of graphs to form a new sparse graph benchmark suite to appraise metaheuristic algorithms for matrix bandwidth reduction. DRSA does not have a local search phase that permits an adequate intensification of the candidate solutions. Another aspect is the three diversification operators employed in its neighborhood permutation. Even using three diversification operators, none necessarily operate on critical vertices. Therefore, DRSA performs many operations without reducing the bandwidth. Hence, the method shows slow convergence [29]. Thus, the time needed to find acceptable results makes the heuristic unsuitable for large-scale instances, recalling that DRSA [24] yielded satisfactory results on the 111 matrices of Martíet al. [69,70] with sizes smaller than 1000. Therefore, there are opportunities for improving the research in this field.

In summary, the developer should devise efficient algorithms to yield even better solutions on large-scale graphs per application field in the bandwidth reduction problem. One can apply different methodologies to the problem, with possibilities of satisfactory results, including Differential Evolution, evolutionary algorithms, reinforcement learning, and hybrid strategies.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Sanderson L. Gonzaga de Oliveira reports financial support was provided by National Council for Scientific and Technological Development. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The author acknowledges the Brazilian funding agency CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico) for financial support with grant 402599/2023-3. The author also thanks the anonymous referees for their careful reading, insightful comments, and suggestions.

Data availability

No data was used for the research described in the article.

References

[1] R.K. Livesley, The analysis of large structural systems, Comput. J. 3 (1) (1960) 34–39.
[2] L.H. Harper, Optimal assignments of numbers to vertices, J. Soc. Ind. Appl. Math. 12 (1964) 131–135.
[3] L.H. Harper, Optimal numberings and isoperimetric problems on graphs, J. Comb. Theory 1 (3) (1966) 385–393, http://dx.doi.org/10.1016/S0021-9800(66)80059-5.
[4] C. Papadimitriou, The NP-completeness of bandwidth minimization problem, Comput. J. 16 (1976) 177–192, http://dx.doi.org/10.1007/BF02280884.
[5] S.L. Gonzaga de Oliveira, C. Carvalho, Metaheuristic algorithms for the bandwidth reduction of large-scale matrices, J. Comb. Optim. 43 (2022) 727–784, http://dx.doi.org/10.1007/s10878-021-00801-6.
[6] M.R. Garey, R.L. Graham, D.S. Johnson, D. Knuth, Complexity results for bandwidth minimization, SIAM J. Appl. Math. 34 (1978) 477–495.
[7] J. Petit, Addenda to the survey of layout problems, Bull. EATCS 105 (2011) 177–201.
[8] M.R. Garey, D.S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, Freeman, New York, 1979.
[9] G. Ausiello, P. Crescenzi, G. Gambosi, V. Kann, A. Marchetti-Spaccamela, M. Protasi, Complexity and Approximation, Springer-Verlag, Berlin, 1999.
[10] D.P. Williamson, D.B. Shmoys, The Design of Approximation Algorithms, Cambridge University Press, Cambridge, 2011.
[11] G.W. Dueck, J. Jeffs, A heuristic bandwidth reduction algorithm, J. Combin. Math. Combin. Comput. (1995) 97–108.
[12] M. Cygan, M. Pilipczuk, Faster exact bandwidth, in: H. Broersma, T. Erlebach, T. Friedetzky, D. Paulusma (Eds.), Graph-Theoretic Concepts in Computer Science, in: Lecture Notes in Computer Science, vol. 5344, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 101–109, http://dx.doi.org/10.1007/978-3-540-92248-3_10.
[13] M. Cygan, M. Pilipczuk, Exact and approximate bandwidth, Theoret. Comput. Sci. 411 (40–42) (2010) 3701–3713, http://dx.doi.org/10.1016/j.tcs.2010.06.018.
[14] M. Cygan, M. Pilipczuk, Even faster exact bandwidth, ACM Trans. Algorithms 8 (1) (2012) 1–14, http://dx.doi.org/10.1145/2071379.207138.
[15] M. Cygan, M. Pilipczuk, Bandwidth and distortion revisited, Discrete Appl. Math. 160 (4–5) (2012) 494–504, http://dx.doi.org/10.1016/j.dam.2011.10.032.
[16] M. Fürer, S. Gaspers, S.P. Kasiviswanathan, An exponential time 2-approximation algorithm for bandwidth, Theoret. Comput. Sci. 511 (2013) 23–31, http://dx.doi.org/10.1016/j.tcs.2013.03.024.
[17] E. Rodriguez-Tello, J.-K. Hao, J. Torres-Jimenez, An effective two-stage simulated annealing algorithm for the minimum linear arrangement problem, Comput. Oper. Res. 35 (2008) 3331–3346, http://dx.doi.org/10.1016/j.cor.2007.03.001.

[18] G.O. Chagas, S.L. Gonzaga de Oliveira, Metaheuristic-based heuristics for symmetric-matrix bandwidth reduction: A systematic review, Procedia Comput. Sci. 51 (2015) 211–220, http://dx.doi.org/10.1016/j.procs.2015.05.229, International Conference On Computational Science, ICCS 2015.
[19] S.L. Gonzaga de Oliveira, G.O. Chagas, A systematic review of heuristics for symmetric-matrix bandwidth reduction: methods not based on metaheuristics, in: Anais do XLVII Simpósio Brasileiro de Pesquisa Operacional, SBPO, Sociedade Brasileira de Pesquisa Operacional, Porto de Galinhas, Ipojuca, PE, Brazil, 2015, pp. 3304–3315, URL http://www.din.uem.br/sbpo/sbpo2015/pdf/142758.pdf.
[20] G.C. Everstine, A comparison of three resequencing algorithms for the reduction of matrix profile and wavefront, Internat. J. Numer. Methods Engrg. 14 (1979) 837–853, http://dx.doi.org/10.1002/nme.1620140606.
[21] N.E. Gibbs, A Survey of Bandwidth and Profile Reduction Algorithms, Tech. Rep. ADA090156, College of William and Mary Williamsburg VA Dept of Mathematics and Computer Science, Williamsburg, USA, 1980, Relatório técnico.
[22] S.L. Gonzaga de Oliveira, A survey of heuristics for profile and wavefront reductions, ACM Comput. Surv. 57 (5) (2025) 1–16, http://dx.doi.org/10.1145/3711120, Article No.: 133.
[23] E.F. Moore, The shortest path through a maze, in: Proceedings of the International Symposium on the Theory of Switching, Harvard University Press, 1959, pp. 285–292.
[24] J. Torres-Jimenez, I. Izquierdo-Marquez, A. Garcia-Robledo, A. Gonzalez-Gomez, J. Bernal, R.N. Kacker, A dual representation simulated annealing algorithm for the bandwidth minimization problem on graphs, Inform. Sci. 303 (2015) 33–49, http://dx.doi.org/10.1016/j.ins.2014.12.041.
[25] S.L. Gonzaga de Oliveira, J.A.B. Bernardes, G.O. Chagas, An evaluation of low-cost heuristics for matrix bandwidth and profile reductions, Comput. Appl. Math. 37 (2018) 1412–1471, http://dx.doi.org/10.1007/s40314-016-0394-9.
[26] S.L. Gonzaga de Oliveira, J.A.B. Bernardes, G.O. Chagas, An evaluation of reordering algorithms to reduce the computational cost of the incomplete Cholesky-conjugate gradient method, Comput. Appl. Math. 37 (2018) 2965–3004, http://dx.doi.org/10.1007/s40314-017-0490-5.
[27] S.L.G. de Oliveira, L.M. Silva, Evolving reordering algorithms using an ant colony hyperheuristic approach for accelerating the convergence of the ICCG method, Eng. Comput. 36 (2020) 1857–1873, http://dx.doi.org/10.1007/s00366-019-00801-5.
[28] P.H.G. Silva, D.N. Brandão, I.S. Morais, S.L.G. de Oliveira, A biased random-key genetic algorithm for bandwidth reduction, in: O. Gervasi, B. Murgante, S. Misra, C. Garau, I. Blečić, D. Taniar, B.O. Apduhan, A.M.A. Rocha, E. Tarantino, C.M. Torre, Y. Karaca (Eds.), Computational Science and Its Applications – ICCSA 2020. ICCSA 2020, in: Lecture Notes in Computer Science, vol. 12249, Springer, Cham, 2020, pp. 312–321, http://dx.doi.org/10.1007/978-3-030-58799-4_23.
[29] S.L. Gonzaga de Oliveira, L.M. Silva, An ant colony hyperheuristic approach for matrix bandwidth reduction, Appl. Soft Comput. 94 (2020) 106434, http://dx.doi.org/10.1016/j.asoc.2020.106434.
[30] S.L. Gonzaga de Oliveira, An evaluation of heuristic methods for the bandwidth reduction of large-scale graphs, Pesqui. Oper. 43 (e268255) (2023) 1–22, http://dx.doi.org/10.1590/0101-7438.2023.043.00268255.
[31] A.A.A.M. Abreu, S.L. Gonzaga de Oliveira, Iterated local search with Tabu Search for the bandwidth reduction problem in graphs, in: O. Gervasi, B. Murgante, C. Garau, D. Taniar, A.M.A. C. Rocha, M.N. Faginas Lago (Eds.), Computational Science and Its Applications – ICCSA 2024. ICCSA 2024, in: Lecture Notes in Computer Science, vol. 14813, Springer, Cham, 2024, pp. 125–136, http://dx.doi.org/10.1007/978-3-031-64605-8_9.
[32] T.A. Davis, Y. Hu, The University of Florida sparse matrix collection, ACM Trans. Math. Software 38 (1) (2011) 1–25, http://dx.doi.org/10.1145/2049662.2049663.
[33] R.L. Burden, J.D. Faires, A.M. Burden, Numerical Analysis, tenth ed., Cengage Learning, Boston, 2016.
[34] J. Scott, M. Tuma, Algorithms for Sparse Linear Systems, Birkhauser, Cham, Switzerland, 2023.
[35] G.G. Alway, D.W. Martin, An algorithm for reducing the bandwidth of a matrix of symmetrical configuration, Comput. J. 8 (3) (1965) 264–272, http://dx.doi.org/10.1093/comjnl/8.3.264.
[36] E. Cuthill, J. McKee, Reducing the bandwidth of sparse symmetric matrices, in: ACM Proceedings of the 1969 24th National Conference, ACM, New York, USA, 1969, pp. 157–172.
[37] J.A. George, Computer Implementation of the Finite Element Method (Ph.D. thesis), Computer Science Department, Stanford University, CA, USA, 1971, p. 228.
[38] N.E. Gibbs, W.G. Poole, P.K. Stockmeyer, An algorithm for reducing the bandwidth and profile of a sparse matrix, SIAM J. Numer. Anal. 13 (2) (1976) 236–250.
[39] A. George, J.W.H. Liu, Computer Solution of Large Sparse Positive Definite Systems, Prentice-Hall, Englewood Cliffs, 1981.
[40] M. Benzi, Preconditioning techniques for large linear systems: a survey, J. Comput. Phys. 182 (2002) 418–477, http://dx.doi.org/10.1006/jcph.2002.7176.
[41] V. Manin, B. Lang, Efficient parallel reduction of bandwidth for symmetric matrices, Parallel Comput. 115 (2023) 102998, http://dx.doi.org/10.1016/j.parco.2023.102998.
[42] R. Rosen, Matrix bandwidth minimization, in: Proceedings of the 23th ACM National Conference, ACM, Brandon Systems Press, Princeton, NJ, 1968, pp. 585–595.
[43] A. George, J.W.H. Liu, An implementation of a pseudoperipheral node finder, ACM Trans. Math. Software 5 (3) (1979) 284–295.
[44] S.L. Gonzaga De Oliveira, A.A.A.M. Abreu, An evaluation of pseudoperipheral vertex finders for the Reverse Cuthill-McKee method for bandwidth and profile reductions of symmetric matrices, in: 2018 37th International Conference of the Chilean Computer Science Society, SCCC, Santiago, Chile, 2018, pp. 1–9, http://dx.doi.org/10.1109/SCCC.2018.8705263.
[45] The MathWorks Inc, MATLAB, 1994–2024, (Access 7 March 2024). URL http://www.mathworks.com/products/matlab/index.html.
[46] J.W. Eaton, D. Bateman, S. Hauberg, R. Wehbring, GNU Octave version 4.0.0 manual: a high-level interactive language for numerical computations, 2015, URL http://www.gnu.org/software/octave/doc/interpreter.
[47] Wolfram, Wolfram mathematica, 2023, wolfram.com/mathematica, (Access 1 January 2023).
[48] ViennaCL, ViennaCL, 2016, viennacl.sourceforge.net, (Access 21 November 2022).
[49] Boost, Boost C++ libraries, 2017, (Access 21 November 2022). URL http://www.boost.org.
[50] SciPy, SciPy - Fundamental algorithms for scientific computing in Python, 2019, scipy.org, (Access 21 November 2022).
[51] Networkx, Networkx - network analysis in Python, 2022, networkx.org, (Access 21 November 2022).
[52] Rust Programming Language, Module sprs::linalg::reverse_cuthill_mckee, 2024, (Access 9 January 2024). URL https://docs.rs/sprs/latest/sprs/linalg/index.html.
[53] A. Esposito, M.S.F. Catalano, F. Malucelli, L. Tarricone, A new matrix bandwidth reduction algorithm, Oper. Res. Lett. 23 (1998) 99–107, http://dx.doi.org/10.1016/S0167-6377(98)00040-6.
[54] A. Esposito, L. Tarricone, Matrix bandwidth reduction with Tabu search: parallel implementation on Cray T3D and applications to microwave circuit design, in: E. Voli (Ed.), Science and Supercomputing at CINECA: Report, CINECA, Bologna, Italy, 1996, pp. 440–450.
[55] S.L. Gonzaga de Oliveira, G.O. Chagas, D.T. Robaina, D.N. Brandão, M. Kischnhevsky, A modified bandwidth reduction heuristic based on the WBRA and George-Liu algorithm, in: Y. Shi, H. Fu, Y. Tian, V.V. Krzhizhanovskaya, M.H. Lees, J. Dongarra, P.M.A. Sloot (Eds.), Computational Science – ICCS 2018. ICCS 2018, in: Lecture Notes in Computer Science, vol. 10862, Springer, Cham, 2018, pp. 416–424, http://dx.doi.org/10.1007/978-3-319-93713-7_35.
[56] B. Koohestani, R. Poli, A hyper-heuristic approach to evolving algorithms for bandwidth reduction based on genetic programming, in: M. Bramer, M. Petridis, L. Nolle (Eds.), Research and Development in Intelligent Systems XXVIII, Springer, London, 2011, pp. 93–106, http://dx.doi.org/10.1007/978-1-4471-2318-7_7.
[57] I.P. King, An automatic reordering scheme for simultaneous equations derived from network systems, Internat. J. Numer. Methods Engrg. 2 (4) (1970) 523–533, http://dx.doi.org/10.1002/nme.1620020406.
[58] M.W. Berry, B. Hendrickson, P. Raghavan, Sparse matrix reordering schemes for browsing hypertext, in: J. Renegar, M. Shub, S. Smale (Eds.), in: Lectures in Applied Mathematics, vol. 32, American Mathematical Society Press, Park City, Utah, USA, 1996, pp. 99–123, The Mathematics of Numerical Analysis.
[59] P. Arbenz, A. Cleary, J. Dongarra, M. Hegland, A comparison of parallel solvers for diagonally dominant and general narrowbanded linear systems, in: J.J. Dongarra, E.J. Kontoghiorghes (Eds.), Parallel Numerical Linear Algebra, Nova Science Publishers, Inc., Commack, Nova York, 2001, pp. 35–56.
[60] D.J. Higham, Unravelling small world networks, J. Comput. Appl. Math. 158 (1) (2003) 61–74, http://dx.doi.org/10.1016/S0377-0427(03)00471-0.
[61] A. Lim, B. Rodrigues, F. Xiao, Heuristics for matrix bandwidth reduction, European J. Oper. Res. (2006) 69–91, http://dx.doi.org/10.1016/j.ejor.2005.02.066.
[62] C. Mueller, B. Martin, A. Lumsdaine, A comparison of vertex ordering algorithms for large graph visualization, in: Proceedings of the 6th International Asia-Pacific Symposium on Visualization, APVIS'07, Sydney, Australia, 2007, pp. 141–148, http://dx.doi.org/10.1109/APVIS.2007.329289.
[63] M.E. Bolanos, S. Aviyente, H. Radha, Graph entropy rate minimization and the compressibility of undirected binary graphs, in: Proceedings of the IEEE Statistical Signal Processing Workshop, SSP, IEEE, Ann Arbor, MI, 2012, pp. 109–112, http://dx.doi.org/10.1109/SSP.2012.6319634.
[64] H.-K. Chen, Evaluation of triangular mesh layout techniques using large mesh simplification, Multimedia Tools Appl. 76 (23) (2017) 25391–25419, http://dx.doi.org/10.1007/s11042-017-4607-z.
[65] E.G. Amparore, M. Beccuti, S. Donatelli, Gradient-based variable ordering of decision diagrams for systems with structural units, in: D. D'Souza, K.N. Kumar (Eds.), ATVA 2017, in: Lecture Notes in Computer Science, vol. 10482, Springer, Cham, 2017, pp. 184–200.
[66] A. Concas, C. Fenu, G. Rodriguez, PQser: a Matlab package for spectral seriation, Numer. Algorithms 80 (3) (2019) 879–902, http://dx.doi.org/10.1007/s11075-018-0510-6.

[67] M. Freire, R. Marichal, S.L. Gonzaga de Oliveira, E. Dufrechou, P. Ezzatti, Enhancing the sparse matrix storage using reordering techniques, in: C.J. Barrios H., S. Rizzi, E. Meneses, E. Mocskos, J.M. Monsalve Diaz, J. Montoya (Eds.), High Performance Computing. CARLA 2023, in: Communications in Computer and Information Science, vol. 1887, Springer, Cham, 2024, pp. 66–76, http://dx.doi.org/10.1007/978-3-031-52186-7_5.

[68] N. Mladenovic, D. Urosevic, D. Pérez-Brito, C.G. García-González, Variable neighbourhood search for bandwidth reduction, European J. Oper. Res. 200 (2010) 14–27, http://dx.doi.org/10.1016/j.ejor.2008.12.015.

[69] R. Martí, M. Laguna, F. Glover, V. Campos, Reducing the bandwidth of a sparse matrix with tabu search, European J. Oper. Res. 135 (2) (2001) 450–459, http://dx.doi.org/10.1016/S0377-2217(00)00325-8.

[70] E. Piñana, I. Plana, V. Campos, R. Martí, GRASP and path relinking for the matrix bandwidth minimization, European J. Oper. Res. 153 (1) (2004) 200–210, http://dx.doi.org/10.1016/S0377-2217(02)00715-4.

[71] A. Lim, J. Lin, F. Xiao, Particle Swarm optimization and Hill Climbing for the bandwidth minimization problem, Appl. Intell. 3 (26) (2007) 175–182, http://dx.doi.org/10.1007/s10489-006-0019-x.

[72] A. Lim, B. Rodrigues, F. Xiao, A fast algorithm for bandwidth minimization, Int. J. Artif. Intell. Tools 16 (3) (2007) 537–544, http://dx.doi.org/10.1142/S0218213007003394.

[73] E. Rodriguez-Tello, J.-K. Hao, J. Torres-Jimenez, An improved simulated annealing algorithm for bandwidth minimization, European J. Oper. Res. 185 (2008) 1319–1335, http://dx.doi.org/10.1016/j.ejor.2005.12.052.

[74] J. Maues, I. Mendonça, G. Amorim, S.L. Gonzaga de Oliveira, A.I. Pereira, D. Brandão, P.H. González, A multi-centrality heuristic for the bandwidth reduction problem, in: O. Gervasi, B. Murgante, C. Garau, D. Taniar, A.M.A.C. Rocha, M. Faginas Lago (Eds.), Computational Science and Its Applications – ICCSA 2024, ICCSA 2024, in: Lecture Notes in Computer Science, vol. 14813, Springer, Cham, 2024, pp. 3–15, http://dx.doi.org/10.1007/978-3-031-64605-8_1.

[75] V. Campos, E. Piñana, R. Martí, Adaptive memory programming for matrix bandwidth minimization, Ann. Oper. Res. 183 (2011) 7–23, http://dx.doi.org/10.1007/s10479-009-0573-9.

[76] S.L. Gonzaga de Oliveira, A.A.A.M. Abreu, D. Robaina, M. Kischnhevsky, An evaluation of four reordering algorithms to reduce the computational cost of the Jacobi-preconditioned Conjugate Gradient Method using high-precision arithmetic, Int. J. Bus. Intell. Data Min. 12 (2) (2017) 190–209, http://dx.doi.org/10.1504/IJBIDM.2017.084281.

[77] S.L. Gonzaga de Oliveira, G.O. Chagas, J.A.B. Bernardes, An analysis of reordering algorithms to reduce the computational cost of the Jacobi-preconditioned CG solver using high-precision arithmetic, in: O. Gervasi, B. Murgante, S. Misra, G. Borruso, C.M. Torre, A.M.A. Rocha, D. Taniar, B.O. Apduhan, E. Stankova, A. Cuzzocrea (Eds.), Computational Science and Its Applications – ICCSA 2017, ICCSA 2017, in: Lecture Notes in Computer Science, vol. 10404, Springer, Cham, 2017, pp. 3–19, http://dx.doi.org/10.1007/978-3-319-62392-4_1.

[78] S.L. Gonzaga de Oliveira, G.O. Chagas, J.A.B. Bernardes, D.T. Robaina, M. Kischinhevsky, An assessment of reordering algorithms to speed up the ICCG method applied to CFD problems, in: O. Gervasi, B. Murgante, S. Misra, E. Stankova, C.M. Torre, A.M.A. Rocha, D. Taniar, B.O. Apduhan, E. Tarantino, Y. Ryu (Eds.), Computational Science and Its Applications – ICCSA 2018, ICCSA 2018, in: Lecture Notes in Computer Science, vol. 10960, Springer, Cham, 2018, pp. 35–52, http://dx.doi.org/10.1007/978-3-319-95162-1_3.

[79] I.S. Duff, R.G. Grimes, J.G. Lewis, Users' Guide for the Harwell-Boeing Sparse Matrix Collection, Tech. rep., Research and Technology Division, Boeing Computer Services, 1992.

[80] J. Petit, Layout Problems (Ph.D. thesis), Departament de Llenguatges i Sistemes Informàtics de la Universitat Politècnica de Catalunya, 2001.

[81] C.E. Leiserson, Area-efficient graph layouts, in: Proceedings of the 21st Annual Symposium Foundation of Computer Science, 1980, pp. 270–281, http://dx.doi.org/10.1109/SFCS.1980.13.

[82] T. Lengauer, Black-white pebbles and graph separation, Acta Inform. 16 (1981) 465–475, http://dx.doi.org/10.1007/BF00264496.

[83] J. Díaz, J. Petit, M. Serna, A survey on graph layout problems, ACM Comput. Surv. 34 (3) (2002) 313–356, http://dx.doi.org/10.1145/568522.56852.

[84] D.J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent, S. Iqbal, J.-B. Lespiau, A. Ahern, T. Köppe, K. Millikin, S. Gaffney, S. Elster, J. Broshear, C. Gamble, K. Milan, R. Tung, M. Hwang, T. Cemgil, M. Barekatain, Y. Li, A. Mandhane, T. Hubert, J. Schrittwieser, D. Hassabis, P. Kohli, M. Riedmiller, O. Vinyals, D. Silver, Faster sorting algorithms discovered using deep reinforcement learning, Nature 618 (2023) 257–263, http://dx.doi.org/10.1038/s41586-023-06004-9.

[85] M.A. Ajiz, A. Jennings, A robust incomplete Choleski-conjugate gradient algorithm, Internat. J. Numer. Methods Engrg. 20 (1984) 949–966, http://dx.doi.org/10.1002/nme.1620200511.